Programmation Système
TP no 1 (Compléments) : Processus, tubes et communications

Guillaume Mercier
email : guillaume.mercier@bordeaux-inp.fr

6 Octobre 2025

1 Client-serveur local

Le but de cet exercice est d'écrire une sorte de programme client-serveur fonctionnant en mode local, c'est-à-dire que les processus clients et serveurs s'exécutent sur la même machine physique. Seuls les moyens de communications locaux sont autorisés pour cet exercice, à l'exception des sockets (domaine `AF_UNIX` ou autre ...). Bien évidemment, vous n'avez pas accès non plus aux primitives de types `connect` et `accept`.

1.1 Hypothèses et données de base

Affiliation des processus Les processus clients et le processus serveur ne sont *pas* affiliés. En revanche, le processus serveur va créer des processus-enfants afin de répondre aux requètes des processus clients. Pour le moment, le système que vous allez mettre en place sera limité car tous les processus (le serveur et les clients) appartiennent à un même utilisateur.

Identification du serveur On supposera que le processus serveur tourne continuellement sur la machine et que son PID est connu de tous les processus clients (dans le cas des communications réseau, on suppose que l'on connait l'adresse IP de la machine sur laquelle tourne le serveur, ainsi que le numéro de port). Ce PID va donc jouer dans notre cas le rôle du numéro de port dans le cas réseau habituel.

Demande de connexions de la part des clients Pour faire sa demande de connexion, un processus client va envoyer au serveur un signal de type `SIGUSR1`, avec deux numéros qui vont permettre d'identifier deux tubes *nommés* qui serviront aux communications exclusives entre le processus-client demandeur et le processus serveur (ou l'un de ses enfants plus exactement). Les tubes nommés seront créés *par le processus client* dans le répertoire `/tmp` de la machine sur laquelle vous travaillez et leur nom devra suivre le format suivant :
`PIPE_<PID du processus client>_numéro1` et `PIPE_<PID du processus client>_numéro2`. Avec `numéro1` et `numéro2` des numéros générés (semi-)aléatoirement. Cependant, ces numéros seront stockés sur des *entiers non signés de 16 bits* (`unsigned short int` ou `uint16_t`, dont la valeur maximale est `USHRT_MAX`).

Afin d'envoyer ces numéros au serveur en même temps que le signal, vous devrez utiliser le traitant de signal étendu (i.e le champ `sa_sigaction` de la structure `struct sigaction`) ainsi que la fonction `sigqueue`.

`sigqueue` permet d'envoyer un entier 32 bits avec le signal, il va donc vous falloir utiliser ces 32 bits afin d'envoyer les deux numéros d'identification des tubes, codés chacun sur 16 bits (rappel : en C, il existe des opérations *bitwise*, et notamment de décalage » et «, ...).

Acceptation de la connexion par le serveur Le serveur devra mettre en place deux traitants de signaux :

— un premier traitant pour la gestion du signal `SIGCHLD` afin de bien éliminer au fur et à mesure les processus-fils zombies correspondants aux processus de service qu'il va créer pour répondre aux requêtes des clients ;

— un second traitant pour la gestion du signal `SIGUSR1` correspondant aux demandes effectives de connexion émanant des processus clients.

Dès qu'il reçoit une demande, le serveur devra créer un processus-fils qui sera chargé de répondre aux demandes du client qu'il sert. Les communications devront utiliser les tubes créés spécialement à cette occasion par le client.

***Optionnel :* mise en place du service** Maintenant que l'infrastructure d'échange est en place, vous pouvez implémenter un service, comme par exemple le cryptage d'un texte :

1. le client envoie le texte à crypter au serveur (ou plutôt au processus de service dédié) ;
2. le processus de service récupère ce texte et le crypte (vous pouvez utiliser le code de César fait l'an dernier en TP de C ...) ;
3. le processus de service renvoie au client le résultat du cryptage.

Bien entendu, il faudra mettre en place un protocle d'échange puisque les données transitent en tant que flux dans les tubes.

2 Signaux

2.1 Signaux : communications en Morse

Cet exercice a pour but de réaliser une communication utilisant le code *morse* (ou un quelconque code similaire) entre deux processus — un client et un serveur — au moyen de signaux. Un résumé du codage morse est disponible dans le fichier `morse.txt` du répertoire `/net/npers/mercier/Cours/IF210/TP/TP1/Materiel`. Pour information, il faut y ajouter les particularités suivantes :

— les signes morses sont séparés par un blanc ;

— les lettres sont séparées par trois blancs ;

— les mots sont séparés par sept blancs.

Questions

— écrivez un programme `tstsig_1.c` qui boucle infiniment et affiche :

  — `'.'` lorsqu'il reçoit un signal `SIGUSR1` ;

  — `'-'` lorsqu'il reçoit un signal `SIGUSR2` ;

  — `' '` lorsqu'il reçoit un signal `SIGALRM`.

Vous utiliserez la fonction `sigaction(3)` pour gérer la mise en place des gestionnaires (*handler*) de signaux. Vous pourrez utiliser la commande `kill(1)` pour envoyer des signaux au processus depuis un terminal.

— écrivez un programme `tstsig_2.c` à partir de `tstsig_1.c` qui — en plus des fonctionnalités de `tstsig_1.c` — se termine en affichant le message "`[over]`" lorsqu'il reçoit un signal `SIGTERM`.
— écrivez un programme `serveur.c` et un programme `client.c` avec les fonctionnalités suivantes :

serveur Le serveur itère en demandant un message (directement en morse, ou en format texte si vous avez le temps de programmer la conversion en fin de TD) et un numéro de `pid`, puis envoie le message au processus client correspondant en utilisant les 4 signaux ci-dessus avec les mêmes conventions de signification.

client Le client est une version étendue de `tstsig_2.c`. Il prend le `pid` du serveur en argument de ligne de commande puis reçoit et affiche un message envoyé par le serveur.

Note : il est indispensable que le message soit transmis de manière fiable.

2.2 Signaux et zombies

Faites le TD du chapitre 6 sur les signaux (i.e la section 6.9 du support de cours), qui vous servira lors du projet Réseau et Système.